

Leakage-Safe Evaluation of an ROI Classifier

Reproduce five-fold model evidence with fold-local synthetic augmentation

Finley Thomson · Monem Moheq · Srinivas Suggu · Jarrod Hwang

CMPT 310 – D100 Introduction to Artificial Intelligence, Summer 2026

This guide shows another CMPT 310 student how to install, run, inspect, and troubleshoot the project's canonical model-evaluation pipeline. The key technique is **keeping synthetic generation inside each training fold**, so every reported test result comes from a real project unseen during fitting or distribution estimation.

Input A 224-row project-level CSV with seven prediction features and Low/Medium/High labels.	Process Stratified five-fold evaluation of a majority baseline and random forests.	Output CSV/JSON metrics, confusion matrices, comparison plot, and reproducibility manifest.
--	--	---

What you will learn

- How to create an isolated Python environment and verify the data contract.
- How to run a fast real-only comparison and the full custom/synthetic experiment.
- Why generating synthetic samples before a train/test split would leak information.
- How to read accuracy, macro-F1, ordinal error, and row-normalized confusion matrices.
- How to diagnose missing modules, missing columns, stale artifacts, and slow custom training.

Prerequisites

Item	Recommended	Check
Python	3.11+; evidence runtime was 3.12.13	<code>python --version</code>
Shell	Terminal, PowerShell, or VS Code terminal	Open at repository root
Disk	At least 500 MB for environment and artifacts	Keep each run in a new output folder
Files	Model/, Data/, requirements.txt	Do not run from inside Model/

Scope: This guide reproduces evaluation from the included final modeling CSV. Historical preprocessing scripts are not a clean raw-to-final pipeline because some upstream files and a local coordinate-conversion module are absent.

1. Set up the environment and verify the data

1 Open the repository root

The directory should contain Model, Data, Artifacts, tests, and requirements.txt.

```
cd CMPT310_Project_Group-14
python --version
```

2 Create and activate a virtual environment

```
python -m venv .venv
# macOS/Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1
```

3 Install the canonical dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The evaluation requires NumPy, pandas, scikit-learn, and Matplotlib. The historical proximity scripts also import haversine, but that package is not needed to evaluate the included final CSV.

4 Run the tests before a model experiment

```
python -m unittest discover -s tests -v
```

Expected result:

```
Ran 5 tests in ...s
OK
```

5 Confirm the modeling table exists

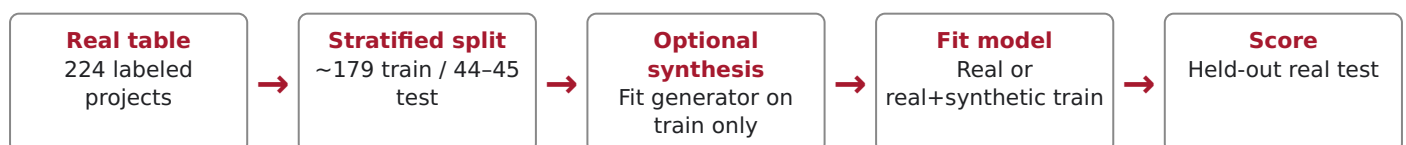
```
Data/CSVs/Sorted/TH_DATA_BY_PROJECT_FINAL.csv
```

The evaluator checks required columns, missing modeling values, consecutive labels, and outcome-column leakage. The seven inputs must retain their exact headers:

- Initial Assessment
- Income
- Cost Per Unit
- Distance To Downtown
- Distance To Nearest Transit Stop
- Distance to Nearest Park
- Distance to Nearest Public School

Leakage rule: Final Assessment, ROI, and Classification are needed for labeling/evaluation but must never be added to FEATURE_COLUMNS.

Data-flow mental model



2. Run the evaluation and find the outputs

1 Run the fast, recommended comparison

```
python -m Model.EvaluateModels \  
  --models dummy sklearn \  
  --output-dir Artifacts/FinalRun
```

This evaluates the majority baseline, real-only scikit-learn random forest, and scikit-learn forest with 3,000 synthetic rows per training fold.

2 Read the terminal summary

A successful run reports the dataset size, class balance, every fold's real/synthetic counts, and the final comparison:

```
Loaded 224 real projects ...  
Classes: Low ROI=75, Medium ROI=74, High ROI=75  
Leakage control: every test fold contains real held-out projects ...  
  
scikit RF (real only): accuracy=0.446±0.085,  
macro-F1=0.444±0.084, ordinal-MAE=0.750,  
severe errors=44  
  
scikit RF (+3000 synthetic): accuracy=0.384±0.048,  
macro-F1=0.306±0.037, ordinal-MAE=0.902,  
severe errors=64
```

3 Optionally run the custom forest

```
python -m Model.EvaluateModels \  
  --models dummy sklearn custom \  
  --output-dir Artifacts/FinalRunAll
```

The custom implementation is slower. For a quick mechanics check, reduce `--trees`; do not compare that reduced run with the saved 100-tree evidence.

4 Inspect the artifact directory

```
Artifacts/FinalRun/ ├── model_comparison.csv ├── model_comparison.png ├── run_manifest.json  
├── scikit_rf_real_only_metrics.json ├── scikit_rf_real_only_fold_metrics.csv ├──  
scikit_rf_real_only_per_class_metrics.csv ├── scikit_rf_real_only_confusion_matrix_counts.png  
└── scikit_rf_real_only_confusion_matrix_normalized.png ... corresponding files for other  
models
```

Reproducibility check: run the same command twice with seed 310 in two new directories, then compare the metric CSV/JSON files. Timestamps and absolute manifest paths can differ; numeric model evidence should match.

Useful command-line variations

Use `--data path/to/file.csv` for another compatible table, `--seed 310` for the deterministic seed, and `--synthetic-samples 0 500 1000 3000` to compare training-set sizes. Always pair the command with a distinct `--output-dir`.

3. Interpret the visual evidence

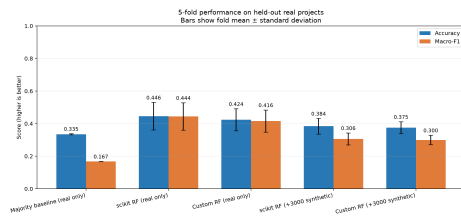


Figure 1. Fold means and standard deviations on real held-out projects.

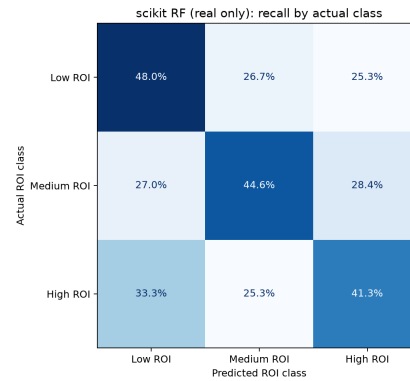


Figure 2. Each row sums to 100%, so the diagonal is class recall.

Read the comparison chart

1. Start with the majority baseline: 33.5% accuracy but 16.7% macro-F1. Predicting only one class can look acceptable under accuracy while failing two classes.
2. Compare real-only forests. The scikit-learn forest is highest at 44.6% accuracy and 44.4% macro-F1; the custom forest is slightly lower.
3. Look at error bars. The selected model varies by about 8.5 percentage points across five folds, so do not present 44.6% as a stable deployment guarantee.
4. Compare synthetic variants. Both decline, so augmentation is a negative result.

Read the normalized confusion matrix

Rows are actual classes and columns are predicted classes. The diagonal shows recall: 48.0% Low, 44.6% Medium, and 41.3% High. The top-right and bottom-left cells are severe two-level errors. The metrics JSON counts 44 severe errors, or 19.6% of the dataset.

Why macro-F1 and ordinal error are useful

Macro-F1

Averages class-specific F1 scores without weighting by support. It exposes the synthetic model's failure to predict Medium cases.

Ordinal MAE

A one-level miss costs 1 and a Low↔High miss costs 2. Lower is better; the selected model scores 0.750 versus baseline 1.000.

Understand the fold-local code

```
for train_indices, test_indices in splitter.split(real_data, y_all):
    train_real = real_data.iloc[train_indices]
    test_real = real_data.iloc[test_indices]

    # Generator sees only this fold's training rows.
    synth_data = generate(train_real, n=synthetic_samples)
    training_data = concat(train_real, synth_data)

    model.fit(training_data[features], training_data[target])
    predictions = model.predict(test_real[features]) # real only
```

If synthetic data were generated once from the complete dataset before splitting, the generator's means/covariance would encode information from future test rows. Keeping generation inside the loop avoids that leakage.

Do not overclaim: 85.0% mean training accuracy versus 44.6% test accuracy is a large overfit signal. Report the result as modest evidence above baseline, not as an accurate investment predictor.

4. Troubleshooting, code samples, and reuse

Symptom	Likely cause	Fix
ModuleNotFoundError: Model	Command started inside Model/ or as a file path.	Return to repository root and run <code>python -m Model.EvaluateModels</code> .
NumPy/pandas/sklearn/Matplotlib missing	Wrong or inactive environment.	Activate <code>.venv</code> ; run <code>python -m pip install -r requirements.txt</code> .
Dataset is missing columns	Headers were renamed/trimmed or an intermediate CSV was supplied.	Use the included final CSV or preserve the exact seven feature and outcome headers.
Modeling columns contain missing values	Input merge left unmatched values.	Audit the source join and imputation decision. Do not silently drop rows after seeing test results.
Custom run takes much longer	Pure-Python tree search across 100 trees and five folds.	Omit custom for a smoke test, or reduce trees only for debugging.
Old and new values appear together	An output directory was reused.	Write each final run to a new empty directory and archive the command/manifest together.
Legacy preprocessing import fails	haversine, upstream CSVs, or local <code>Coordinate_Conversions</code> are absent.	Use the canonical included final CSV; repair/document legacy inputs before claiming raw-to-final reproducibility.

Minimal metrics code sample

```
from sklearn.metrics import accuracy_score, f1_score

accuracy = accuracy_score(y_true, y_pred)
macro_f1 = f1_score(
    y_true, y_pred,
    labels=[0, 1, 2],
    average="macro",
    zero_division=0,
)
ordinal_mae = abs(y_pred - y_true).mean()
```

Final evidence checklist

- Save the exact command, dataset hash, seed, package versions, fold metrics, and confusion matrices.
- Keep held-out evidence real; label any synthetic rows as training-only.
- Report the baseline, variance, per-class behavior, generalization gap, and severe errors.
- Do not expose project addresses, applicant names, email, or SFU student numbers in screenshots.
- Use the real-only forest as the current best model and describe synthetic augmentation as unsuccessful.

References

1. scikit-learn developers. [Cross-validation: evaluating estimator performance](#).
2. scikit-learn developers. [Random forests and other randomized tree ensembles](#).
3. City of Calgary. [Building Permits and Historical Property Assessments](#).
4. Group 14 project source: `Model/EvaluateModels.py`, `Model/CrossValidation.py`, `Model/Evaluation.py`, and `Data/Code/SyntheticData.py`.

Originality and permission statement. This guide was written for this project and does not reproduce an instructor or external guide. Short API concepts are cited above. The authors retain copyright and grant Simon Fraser University permission to share this guide for academic use.